嵌入式系统作业 2

彭俊杰　智装备 241　2024306100324

第 1 题 英文缩写

题目：说明下列英文缩写的含义：ARM、AHB、AHB-Lite、APB、PPB、DAP（JTAG、SWD）、MPU、FPU、NVIC、ISR、Bitband、SCS、SysTick、MSP、PC、SP、GPIO、TIM、UART、CANBUS。

缩写	英文名称	含义
ARM	Advanced RISC Machines	处理器体系结构与内核技术名称；早期源于 Acorn RISC Machine。
AHB	Advanced High-performance Bus	AMBA 高性能总线，连接处理器、存储器和高速外设。
AHB-Lite	Advanced High-performance Bus Lite	AHB 的单主设备简化接口，省去多主设备仲裁信号。
APB	Advanced Peripheral Bus	面向低带宽外设的简单总线，常经 AHB–APB 桥接入。
PPB	Private Peripheral Bus	内核私有外设总线，访问内核控制与调试部件。
DAP	Debug Access Port	调试访问端口，通过 JTAG 或 SWD 访问目标系统。
JTAG	Joint Test Action Group	IEEE 1149.1 边界扫描接口，也用于芯片调试。
SWD	Serial Wire Debug	串行线调试，核心信号为 SWDIO 和 SWCLK。
MPU	Memory Protection Unit	存储器保护单元，设置区域访问权限和属性。
FPU	Floating-Point Unit	浮点运算单元：Cortex-M4F 支持单精度浮点。
NVIC	Nested Vectored Interrupt Controller	嵌套向量中断控制器，管理中断使能、挂起和优先级。
ISR	Interrupt Service Routine	中断服务程序，响应并处理对应事件。
Bitband	Bit-banding	位带机制，将一个位映射到别名区的一个 32 位字。
SCS	System Control Space	系统控制空间，包含 SCB、NVIC、SysTick 等寄存器。
SysTick	System Tick Timer	内核 24 位递减定时器，可产生周期性时基中断。
MSR	Move to Special Register	写特殊寄存器的指令，如 MSR MSP, R0；MRS 为读取。
MSP	Main Stack Pointer	主栈指针；复位后使用，异常处理模式固定使用。
PC	Program Counter	程序计数器，即 R15，表示指令执行位置。
SP	Stack Pointer	栈指针，即 R13；线程模式可选择 MSP 或 PSP。
GPIO	General-Purpose Input/Output	通用输入输出端口，用于数字电平输入和输出。
TIM	Timer	定时器，可用于计时、计数、输入捕获和 PWM。
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发器；USART 还支持同步模式。
CANBUS	Controller Area Network Bus	控制器局域网总线，支持多节点通信与逐位仲裁。

资料来源：[1, 2]。

第 2 题 存储器映射与外设地址

题目：简要画图说明 ARM Cortex-M4 对存储器映射如何划分地址区域的，（自学）标出 GPIOs、SysTick、CAN 这些外设寄存器所在的地址在哪个区域？

Cortex-M4 的 32 位地址空间为 0x00000000～0xFFFFFFFF，共 4 GB，主要分区见图 1。[1]



图 1 Cortex-M4 统一地址空间

对象	寄存器地址或基址	所属区域
GPIOA	0x40020000	外设区 → AHB1
GPIOF	0x40021400	外设区 → AHB1
CAN1 / CAN2	0x40006400 / 0x40006800	外设区 → APB1
SysTick	0xE000E010～0xE000E01C	系统区 → PPB → SCS

STM32F407ZGT6 的 1 MB Flash 位于 0x08000000～0x080FFFFF；SRAM1 与 SRAM2 合计 128 KB，位于 0x20000000～0x2001FFFF；64 KB CCM 数据 RAM 位于 0x10000000～0x1000FFFF，仅供 CPU 通过 D-bus 访问。[2]

位带寻址

SRAM 区和外设区的前 1 MB 支持位带访问，对应别名区基址分别为 0x22000000 和 0x42000000。别名地址 = 别名区基址 + 字节偏移 × 32 + 位号 × 4，其中字节偏移从位带区基址算起。向别名地址写入 0 或 1，即可清零或置位对应的位。[2]

第 3 题 中断概念与异常向量表

题目：画简图说明有关中断的概念（罗列关键词），并说明 ARM Cortex-M4 微处理器异常/中断向量表的作用是什么？

处理器响应中断时，暂停当前程序，执行中断服务程序，结束后恢复执行。Cortex-M4 将外部中断与故障、SVC、PendSV、SysTick 等一并纳入异常机制。[1]

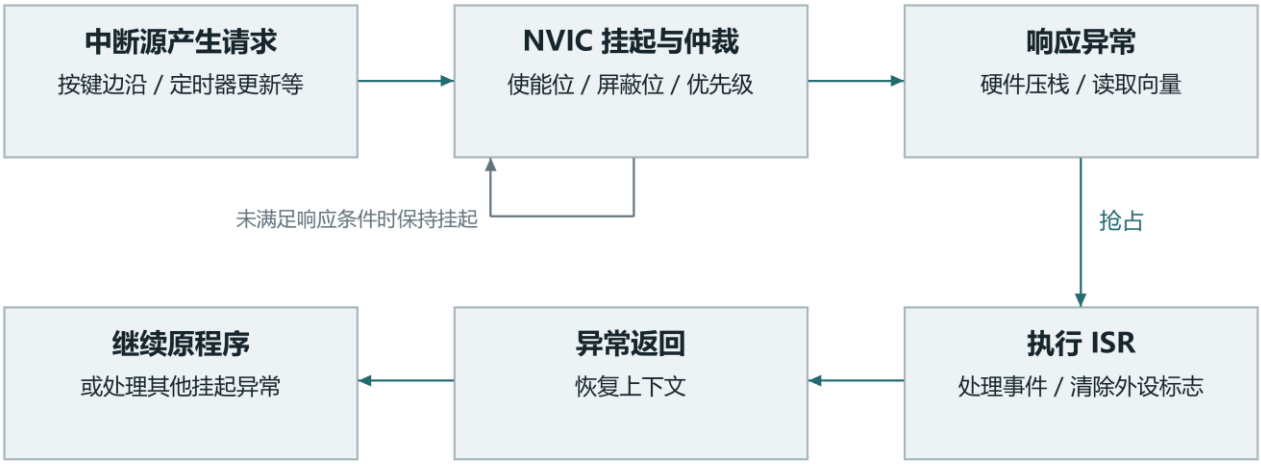


图 2 中断响应过程

关键词：中断源、使能、屏蔽、挂起、优先级、抢占、嵌套、向量、ISR、异常返回。优先级数值越小，优先级越高；高抢占优先级的异常可以打断低优先级的处理程序。外设标志通常由 ISR 按该外设规则清除。[1, 2]

向量表的作用

向量表保存异常处理函数的入口地址，每项占 4 字节。第 0 项保存初始 MSP，第 1 项保存复位入口，其余为异常入口或保留项。外部中断 IRQn 的表项偏移为 4 × (16 + IRQn)，VTOR 保存向量表基址。[1]

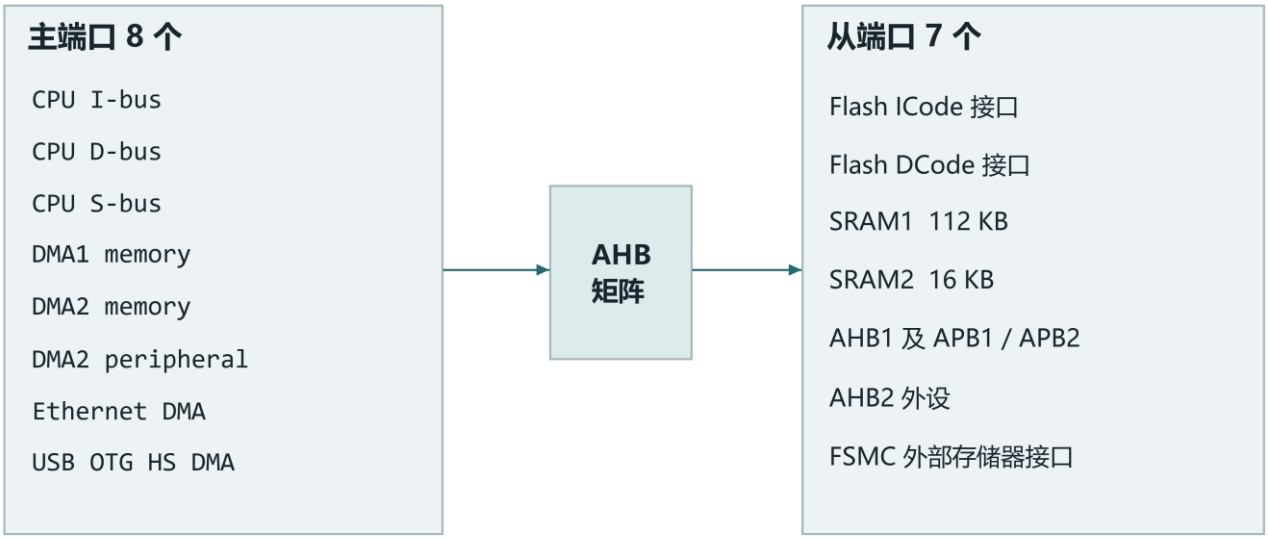
表项	偏移	内容
0	0x00	初始主栈指针 __initial_sp
1	0x04	Reset_Handler
2 / 3	0x08 / 0x0C	NMI_Handler / HardFault_Handler
11 / 14	0x2C / 0x38	SVC_Handler / PendSV_Handler
15	0x3C	SysTick_Handler
16 + IRQn	0x40 起	外部中断入口，例如 WWDG_IRQHandler

异常进入时，基本栈帧保存 R0～R3、R12、LR、PC 和 xPSR。STM32F407 启动文件的向量表共 98 项：16 个系统表项和 82 个外部中断表项，外部中断编号为 IRQ0～IRQ81。[1, 2, 7]

第 4 题 STM32F407 片上总线

题目：列写 STM32F40x 系列微控制器内部结构中片上外设主从总线有哪些，并说明 GPIO、TIMER2（或简写 TIM2）、USART1 模块分别挂在哪条总线上？

STM32F407 通过 32 位多层 AHB 总线矩阵连接处理器、DMA、存储器和外设。主端口发起传输，从端口响应访问；RM0090 列出 8 个主端口和 7 个从端口。[2，第 59～60 页]



CCM 64 KB：CPU D-bus 专用访问；DMA1 外设端口直接连接 APB1。

图 3 STM32F407 总线矩阵及端口

外设连接

模块	挂接总线	基址与用途
GPIO	AHB1	GPIOA 基址 0x40020000；通用输入输出
TIM2	APB1	0x40000000；32 位通用定时器
USART1	APB2	0x40011000；同步或异步串行通信

AHB1 还连接 RCC、CRC、DMA，并经总线桥连接 APB1 和 APB2。AHB2 连接 USB OTG FS、DCMI、RNG，外部存储器经 FSMC 接入。使用 GPIO 前，需在 RCC 中打开对应的 AHB1 时钟使能位。[2]

第 5 题 时钟源与系统时钟

题目：STM32F40x 系列微控制器的高速和低速时钟源指的是哪些？如何得到系统时钟 SYSCLK？AHB1、APB1、APB2 总线的时钟最高频率各是多少？

类别	时钟源	频率与用途
高速内部	HSI	16 MHz RC；复位后的系统时钟来源
高速外部	HSE	4～26 MHz 晶体；本例采用 8 MHz
低速内部	LSI	标称约 32 kHz RC；可供独立看门狗、RTC 使用
低速外部	LSE	32.768 kHz 晶体；常用作 RTC 时钟
倍频输出	主 PLL	输入选择 HSI 或 HSE；经 M、N、P 得到 PLLCLK

SYSCLK 可以直接选择 HSI、HSE，或选择主 PLL 输出。采用 PLL 时，SYSCLK = f 输入 ÷ M × N ÷ P。例如 8 MHz ÷ 8 × 336 ÷ 2 = 168 MHz。[2, 3]

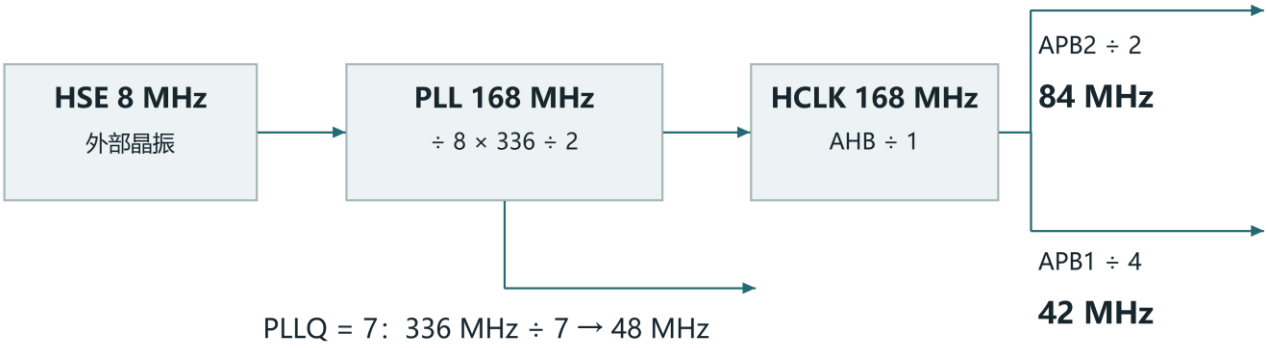


图 4 168 MHz 配置示例

时钟	最高频率	示例配置
SYSCLK	168 MHz	PLL 输出 168 MHz
AHB1 / HCLK	168 MHz	SYSCLK ÷ 1
APB1 / PCLK1	42 MHz	HCLK ÷ 4
APB2 / PCLK2	84 MHz	HCLK ÷ 2

切换到 168 MHz 前，需设置供电档位、Flash 等待周期和总线分频，并等待 PLL 就绪。供电为 2.7～3.6 V 时，Flash 需配置 5 个等待周期。[2，表 11]

APB 分频系数为 1 时，定时器时钟等于 PCLK；大于 1 时，定时器时钟为 2 × PCLK。本例中 APB1 定时器为 84 MHz，APB2 定时器为 168 MHz。[2]

第 6 题 复位引脚与典型电路

题目：在 STM32F407ZGT6 芯片引脚上找到复位脚，说明其名称的含义，并画出典型的复位电路。

STM32F407ZGT6 采用 LQFP144 封装，复位引脚为第 25 脚 NRST。N 表示低电平有效，RST 表示复位；将 NRST 拉低即可复位芯片。[3，表 7]

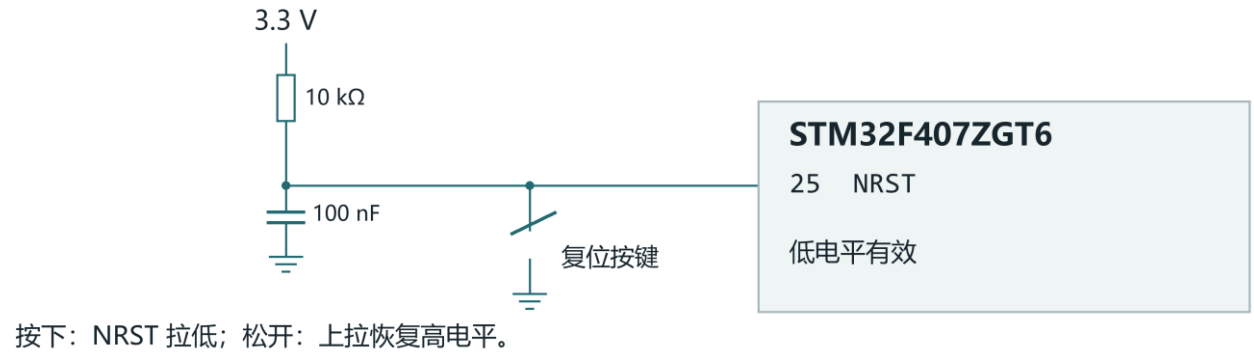


图 5 NRST 典型复位电路

按下按键，NRST 接地，芯片进入复位；松开后恢复高电平，处理器读取初始栈指针和复位入口，重新执行程序。[2, 3]

元件或信号	作用
10 kΩ 上拉电阻	让复位脚在按键松开时保持高电平。
100 nF 电容	与上拉网络配合抑制复位脚上的短暂干扰。
常开按键	闭合时将 NRST 接地，实现手动复位。
调试器 NRST	连接同一复位网络，允许调试器控制复位。

图 5 取自普中 F407-T100 原理图。NRST 内部弱上拉电阻典型值为 40 kΩ。[3，第 121 页；5]

第 7 题 主芯片数据手册与特征

题目：请查找我们实验所用两种开发板（407 通用、潘多拉物联网）的主芯片 STM32F407ZGT6、STM32L496 的 PDF 资料，简单描述一下其特征。

STM32F407ZGT6 与 STM32L496VET6 均采用 Cortex-M4 内核，带单精度 FPU，支持 DSP 指令。[3, 4]

项目	STM32F407ZGT6	STM32L496VET6
官方手册	DS8626 Rev 12 STM32F405xx / STM32F407xx	DS11585 Rev 20 STM32L496xx
CPU 最高频率	168 MHz	80 MHz
Flash	1 MB	512 KB
SRAM	192 KB，其中 CCM 为 64 KB；另有 4 KB 备份 SRAM	320 KB，其中 64 KB 带硬件奇偶校验
封装	LQFP144	LQFP100
工作电源	1.8～3.6 V	1.71～3.6 V
主要接口	以太网 MAC、USB OTG FS/HS、CAN、USART、SPI、I²C、FSMC	USB OTG FS、CAN、LPUART、I²C、SPI、QUADSPI、SDMMC
模拟与控制	3 个 12 位 ADC、2 个 DAC 通道、多组 TIM	3 个 12 位 ADC、2 个 DAC 通道、运放、比较器、低功耗定时器
低功耗模式	Sleep、Stop、Standby	Sleep、低功耗运行、低功耗睡眠、Stop 0/1/2、Standby、Shutdown

L496 系列最高提供 1 MB Flash，VET6 型号为 512 KB。F407 使用以太网时需外接 PHY，USB OTG HS 工作在高速模式时需外接 ULPI PHY。[3, 4]

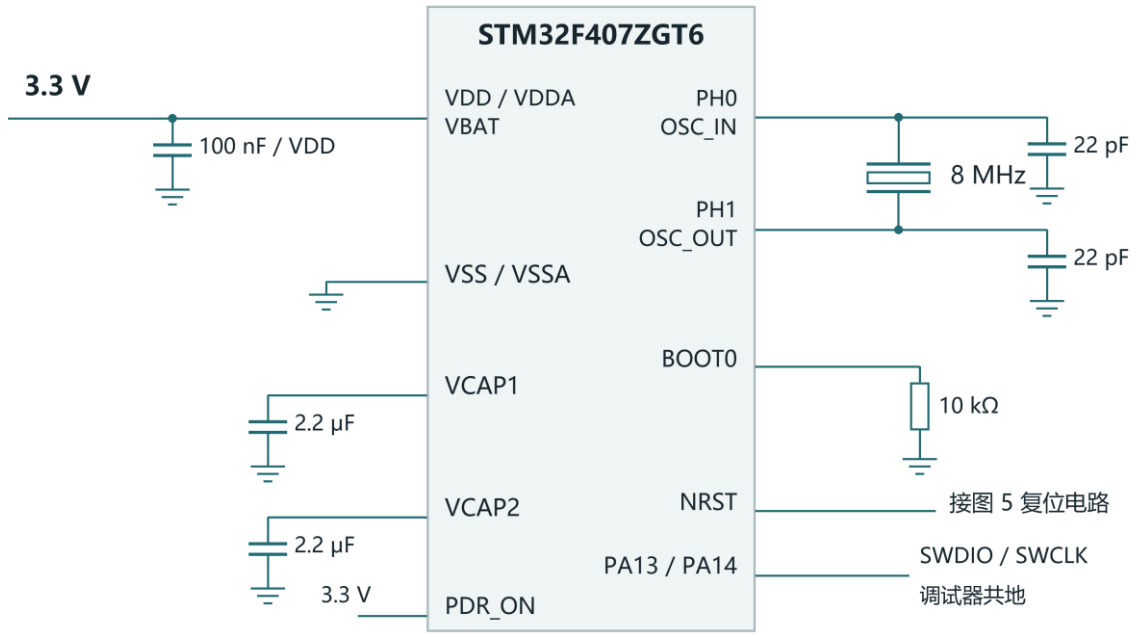
开发板与芯片型号的对应

普中 F407-T100 采用 STM32F407ZGT6。正点原子潘多拉板的公开 BSP 标注为 STM32L475VET6，主频 80 MHz，含 512 KB Flash 和 128 KB RAM；STM32L496 的参数见上表。[5, 6]

第 8 题 扩展练习一 最小系统电路

题目：①查阅淘宝上 STM32F407ZGT6 最小系统的电路图，贴出来。

图 6 采用 3.3 V 供电和 8 MHz 外部晶振，包括电源去耦、复位、启动配置及 SWD 调试接口。[3, 5, 8]



VBAT 无电池时接 3.3 V；电源入口另设储能电容。
SWD 接口引出 3.3 V 参考、GND、SWDIO、SWCLK 和 NRST。

图 6 STM32F407ZGT6 最小系统连接示意，据原理图与硬件指南绘制。[5, 8]

VDD 与 VDDA 接 3.3 V，VSS 与 VSSA 接地。每个 VDD 附近放置 100 nF 去耦电容，VCAP1、VCAP2 各接 2.2 µF 电容到地，用于稳定片内调压器。PDR_ON 接高电平，启用内部复位电路。[3, 8]

BOOT0 下拉时从用户 Flash 启动。PA13 和 PA14 分别用于 SWDIO 与 SWCLK；调试器同时连接 GND、3.3 V 参考和 NRST。RTC 若需要外部低速时钟，可在 PC14、PC15 之间增加 32.768 kHz 晶体及匹配电容。[2, 3]

第 8 题 扩展练习二 工程模板

题目：②参考网络教学平台的资源（安装软件和录制视频），构建标准库和抽象库的空白工程，观察 group、main.c 等信息，理解模板的作用。

Group 用于在 µVision 中分组管理源文件，main.c 包含初始化代码和主循环。工程模板保存芯片型号、头文件路径、宏定义、存储器布局和编译选项，便于在相同配置下添加应用代码。

SPL_GPIO 和 HAL_GPIO 目录各有两个工程：Blank.uvprojx 编译 User/main_blank.c，作为空白模板；Template.uvprojx 编译 User/main.c，作为 GPIO 示例。

组成	标准库工程	HAL 工程
用户程序	main_blank.c 或 main.c 及中断处理文件	main_blank.c 或 main.c 及中断处理文件
启动与内核	startup_stm32f40_41xxx.s CMSIS 与 system_stm32f4xx.c	startup_stm32f407xx.s CMSIS 与 system_stm32f4xx.c
驱动库	STM32F4xx_StdPeriph_Driver	STM32F4xx_HAL_Driver
空白模板入口	main → while (1)	HAL_Init → SystemClock_Config → while (1)

第 8 题 扩展练习三 启动文件

题目：③打开“startup_stm32f40_41xxx.s”或 startup_stm32f407xx.s 文件，观察有关向量表、堆栈等信息，理解复位序列。

SPL 启动文件设置 Stack_Size = 0x00000400，即 1 KB 栈；Heap_Size = 0x00000200，即 512 B 堆。SPACE 保留空间，DCD 放置 32 位数据或地址。向量表和复位入口节选如下。[7]

```
__Vectors       DCD     __initial_sp
                DCD     Reset_Handler
                DCD     NMI_Handler
                DCD     HardFault_Handler

Reset_Handler   PROC
                EXPORT  Reset_Handler [WEAK]
                IMPORT  SystemInit
                IMPORT  __main
                LDR     R0, =SystemInit
                BLX     R0
                LDR     R0, =__main
                BX      R0
                ENDP
```

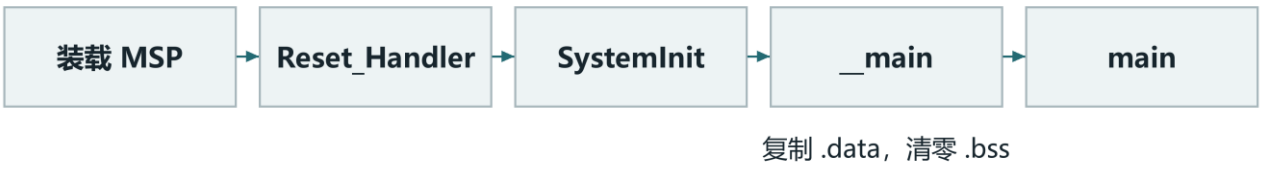


图 7 从复位到 main 的执行顺序

复位后，硬件读取初始 MSP 和复位向量，进入 Reset_Handler。SystemInit 初始化系统，__main 建立 C 运行环境，再调用 main。启动文件中的弱定义中断函数可由用户编写的同名函数覆盖。[1, 7]

第 8 题 扩展练习四 标准库 GPIO 工程

题目：④新建一个 GPIO 的工程，体会标准库和抽象库中初始化、读引脚、写引脚函数的使用。

PF9 接低电平点亮的 LED，PE4 接按下接地的按键。先开启端口时钟，将 PF9 配为推挽输出、PE4 配为上拉输入，再循环读取按键并控制 LED。[5, 7]

代码 1 SPL_GPIO/User/main.c 完整内容

```
#include "stm32f4xx.h"

static void GPIO_Setup(void)
{
    GPIO_InitTypeDef gpio;
    RCC_AHB1PeriphClockCmd(
        RCC_AHB1Periph_GPIOF | RCC_AHB1Periph_GPIOE, ENABLE);

    GPIO_SetBits(GPIOF, GPIO_Pin_9);
    GPIO_StructInit(&gpio);
    gpio.GPIO_Pin = GPIO_Pin_9;
    gpio.GPIO_Mode = GPIO_Mode_OUT;
    gpio.GPIO_OType = GPIO_OType_PP;
    gpio.GPIO_Speed = GPIO_Speed_2MHz;
    gpio.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_Init(GPIOF, &gpio);

    gpio.GPIO_Pin = GPIO_Pin_4;
    gpio.GPIO_Mode = GPIO_Mode_IN;
    gpio.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(GPIOE, &gpio);
}

int main(void)
{
    GPIO_Setup();
    while (1)
    {
        if (GPIO_ReadInputDataBit(GPIOE, GPIO_Pin_4) == Bit_RESET)
            GPIO_ResetBits(GPIOF, GPIO_Pin_9);
        else
            GPIO_SetBits(GPIOF, GPIO_Pin_9);
    }
}
```

GPIO_StructInit 填入默认配置，GPIO_Init 按修改后的参数初始化端口。GPIO_ReadInputDataBit 读取 PE4，GPIO_SetBits 和 GPIO_ResetBits 分别将 PF9 置高和置低。

按下按键时 PE4 为 0，程序将 PF9 清零，LED 点亮；松开时 PE4 被上拉为 1，PF9 置高，LED 熄灭。

第 8 题 扩展练习四 HAL GPIO 工程

题目：④新建一个 GPIO 的工程，体会标准库和抽象库中初始化、读引脚、写引脚函数的使用。

HAL 工程沿用 PF9 和 PE4 的接线。main 先调用 HAL_Init，随后由 SystemClock_Config 将 8 MHz HSE 经 PLL 配为 168 MHz，再初始化 GPIO。

代码 2 HAL_GPIO/User/main.c 中的初始化与主循环

```
static void GPIO_Setup(void)
{
    GPIO_InitTypeDef gpio = {0};
    __HAL_RCC_GPIOF_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();

    HAL_GPIO_WritePin(GPIOF, GPIO_PIN_9, GPIO_PIN_SET);
    gpio.Pin = GPIO_PIN_9;
    gpio.Mode = GPIO_MODE_OUTPUT_PP;
    gpio.Pull = GPIO_NOPULL;
    gpio.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOF, &gpio);

    gpio.Pin = GPIO_PIN_4;
    gpio.Mode = GPIO_MODE_INPUT;
    gpio.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOE, &gpio);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    GPIO_Setup();
    while (1)
    {
        GPIO_PinState key = HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_4);
        HAL_GPIO_WritePin(GPIOF, GPIO_PIN_9, key);
    }
}
```

HAL_GPIO_ReadPin 返回 PE4 的电平，HAL_GPIO_WritePin 将该电平写到 PF9，得到按下点亮、松开熄灭的效果。初始化时先写高电平，再将 PF9 配为输出，使 LED 初始熄灭。

操作	标准库	HAL
打开时钟	RCC_AHB1PeriphClockCmd	__HAL_RCC_GPIOx_CLK_ENABLE
初始化	GPIO_Init	HAL_GPIO_Init
读输入	GPIO_ReadInputDataBit	HAL_GPIO_ReadPin
写输出	GPIO_SetBits / GPIO_ResetBits	HAL_GPIO_WritePin

编译记录与参考资料

编译环境为 Keil µVision，编译器为 Arm Compiler 5.06 update 7 build 960。四个工程的编译结果如下。

工程	编译结果	代码与只读数据
SPL_Blank	0 错误，0 警告	Code 704 B，RO-data 408 B
HAL_Blank	0 错误，0 警告	Code 3106 B，RO-data 450 B
SPL_GPIO	0 错误，0 警告	Code 1068 B，RO-data 408 B
HAL_GPIO	0 错误，0 警告	Code 3814 B，RO-data 450 B

参考资料

[1] STMicroelectronics. PM0214 Rev 10. Cortex-M4 编程手册。 原文

[2] STMicroelectronics. RM0090 Rev 22. STM32F405/407 等器件参考手册。 文档入口

[3] STMicroelectronics. DS8626 Rev 12. STM32F405xx / STM32F407xx 数据手册，2026 年 3 月。 原文

[4] STMicroelectronics. DS11585 Rev 20. STM32L496xx 数据手册，2025 年 10 月。 原文

[5] 普中. F407-T100 开发板原理图，2020 年 12 月 24 日。见附件“参考资料”。

[6] RT-Thread. STM32L475 潘多拉开发板 BSP 说明。 原文

[7] SPL 与 HAL 空白模板及 GPIO 示例。见附件“GPIO 工程”；模板分别取自普中例程和本地 HAL 最小工程。

[8] STMicroelectronics. AN4488. Getting started with STM32F4xxxx MCU hardware development。 原文